APM1110_SA1_DeRamos_AneezaAnn_and_Eduvala_Mikhael

Aneeza Ann de Ramos and Mikhael Eduvala

2026-09-27

```r
# Helper function for input prompt with fallback default for knit compatibility
get_input <- function(prompt_text, default_val) {
  if (interactive()) {
    ans <- as.numeric(readline(prompt = prompt_text))
    if (is.na(ans)) return(default_val) else return(ans)
  } else {
    return(default_val)
  }
}

# Production proportions (x1, x2, x3)
x1 <- get_input("Enter x1 (0.10 to 0.40): ", 0.30)
x2 <- get_input("Enter x2 (0.10 to 0.40): ", 0.35)
x3 <- get_input("Enter x3 (0.10 to 0.40): ", 0.35)

# Defect rates (y1, y2, y3)
y1 <- get_input("Enter y1 (0.01 to 0.05): ", 0.03)
y2 <- get_input("Enter y2 (0.01 to 0.05): ", 0.04)
y3 <- get_input("Enter y3 (0.01 to 0.05): ", 0.05)

# Input Validation
if (!isTRUE(all.equal(sum(x1, x2, x3), 1))) {
  stop("Error: x1 + x2 + x3 must sum to 1.")
}
if (!isTRUE(all.equal(sum(y1, y2, y3), 0.12))) {
  stop("Error: y1 + y2 + y3 must sum to 0.12.")
}
if (any(c(x1, x2, x3) < 0.10) || any(c(x1, x2, x3) > 0.40)) {
  stop("Error: x1, x2, and x3 must be between 0.10 and 0.40.")
}
if (any(c(y1, y2, y3) < 0.01) || any(c(y1, y2, y3) > 0.05)) {
  stop("Error: y1, y2, and y3 must be between 0.01 and 0.05.")
}

# Total probability of defect using Law of Total Probability
P_defective <- x1 * y1 + x2 * y2 + x3 * y3
cat("Total Probability of a Defective Product P(D):", P_defective, "\n")
```

```
## Total Probability of a Defective Product P(D): 0.0405
```

Interpretation The overall probability of selecting a defective product across all three manufacturing facilities is calculated via the Law of Total Probability: $P(D) = \sum_{i=1}^{3} P(F_i)P(D|F_i)$. With factory production

shares$x = (0.3, 0.35, 0.35)$and defect rates$y = (0.03, 0.04, 0.05)$, the expected overall defect probability.

```r
# Probability validation helper
validate_probs <- function(probs) {
  return(all(probs >= 0 & probs <= 1) && isTRUE(all.equal(sum(probs), 1)))
}

# --- Part A: Univariate Discrete Random Variable ---
x_vals <- c(1, 2, 3, 4, 5)
p_x <- c(0.1, 0.2, 0.3, 0.2, 0.2)

if (!validate_probs(p_x)) stop("Invalid univariate probabilities!")

mean_x <- sum(x_vals * p_x)
var_x <- sum(((x_vals - mean_x)^2) * p_x)

cat("Univariate Mean E[X]:", mean_x, "\n")
```

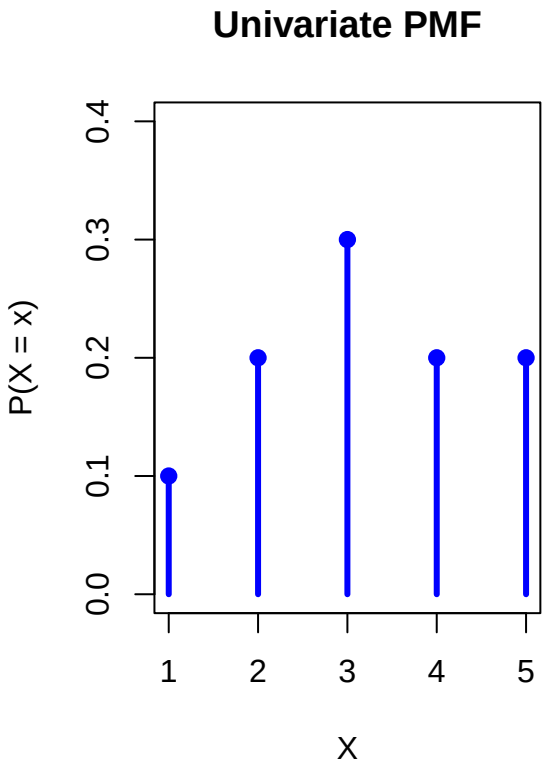
```
## Univariate Mean E[X]: 3.2
```

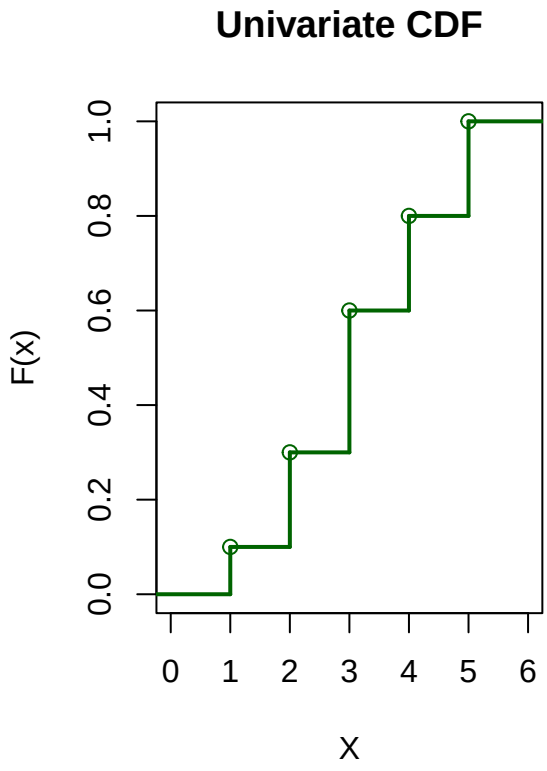
```r
cat("Univariate Variance Var(X):", var_x, "\n")
```

```
## Univariate Variance Var(X): 1.56
```

```r
# PMF and CDF Plots
par(mfrow = c(1, 2))
plot(x_vals, p_x, type = "h", lwd = 3, col = "blue",
     main = "Univariate PMF", xlab = "X", ylab = "P(X = x)", ylim = c(0, 0.4))
points(x_vals, p_x, pch = 16, col = "blue", cex = 1.2)

cdf_x <- cumsum(p_x)
plot(stepfun(x_vals, c(0, cdf_x)), main = "Univariate CDF",
     xlab = "X", ylab = "F(x)", col = "darkgreen", lwd = 2)
```






```r
# --- Part B: Bivariate Discrete Random Variable ---
x_grid <- c(1, 2)
y_grid <- c(1, 2)

# Joint probability matrix (X in rows, Y in columns)
joint_matrix <- matrix(c(0.2, 0.3,
                         0.1, 0.4), nrow = 2, byrow = TRUE)

if (!validate_probs(as.vector(joint_matrix))) stop("Invalid bivariate probabilities!")

# Marginal Distributions
p_x_marg <- rowSums(joint_matrix)
p_y_marg <- colSums(joint_matrix)

# Bivariate Means and Variances
mean_bx <- sum(x_grid * p_x_marg)
var_bx <- sum((x_grid - mean_bx)^2) * p_x_marg)

mean_by <- sum(y_grid * p_y_marg)
var_by <- sum((y_grid - mean_by)^2) * p_y_marg)

# Conditional Probability Distributions
cond_y_given_x <- sweep(joint_matrix, 1, p_x_marg, "/")
cond_x_given_y <- sweep(joint_matrix, 2, p_y_marg, "/")

cat("\nMarginal P(X):", p_x_marg, "\n")
```

```
##
## Marginal P(X): 0.5 0.5
```

```
cat("Marginal P(Y):", p_y_marg, "\n")
```

```
## Marginal P(Y): 0.3 0.7
```

```
cat("Mean X:", mean_bx, "| Variance X:", var_bx, "\n")
```

```
## Mean X: 1.5 | Variance X: 0.25
```

```
cat("Mean Y:", mean_by, "| Variance Y:", var_by, "\n")
```

```
## Mean Y: 1.7 | Variance Y: 0.21
```

```
cat("\nConditional P(Y | X):\n")
```

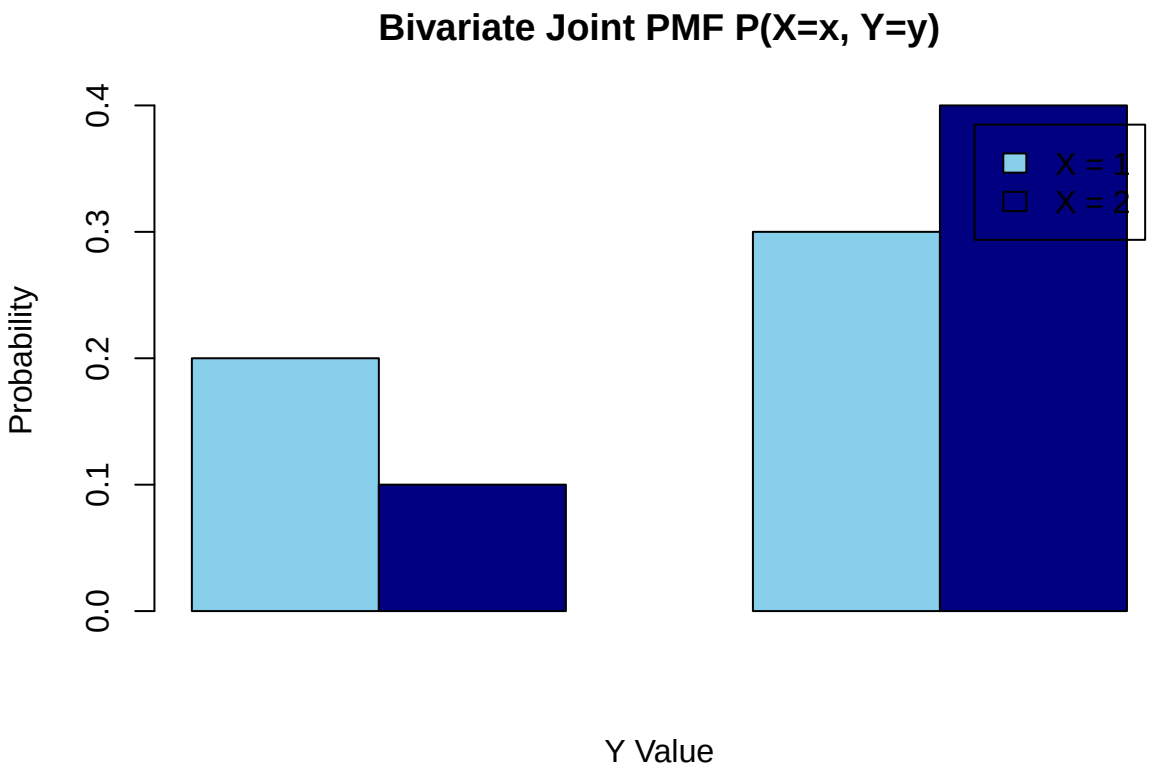
```
##
## Conditional P(Y | X):
```

```
print(cond_y_given_x)
```

```
##      [,1] [,2]
## [1,]  0.4  0.6
## [2,]  0.2  0.8
```

```
# 3D Joint PMF Plot
par(mfrow = c(1, 1))
joint_cdf <- apply(joint_matrix, 1, cumsum)
joint_cdf <- t(apply(joint_cdf, 1, cumsum))

barplot(joint_matrix, beside = TRUE, col = c("skyblue", "navy"),
        legend.text = c("X = 1", "X = 2"),
        main = "Bivariate Joint PMF P(X=x, Y=y)", xlab = "Y Value", ylab = "Probability")
```



Interpretation **Univariate Variable*:** The calculated expected value $E[X] = 3.2$represents the center of mass of the distribution, with variance$Var(X) = 1.56$ measuring dispersion.

Bivariate Variable: The marginal distribution sums joint probabilities across rows and columns. The conditional matrix $P(Y|X)$ shows how the distribution of Y shifts given specific observed values of X.

```
set.seed(42) # Ensure reproducible simulation results
n_trials <- 10000
p <- 0.60 # Probability site contains key phrase

# Generate Geometric simulation (number of trials until first success)
# rgeom yields number of failures before success, so add 1 for total searches
searches <- rgeom(n_trials, prob = p) + 1

# Complete Set Metrics
sim_mean <- mean(searches)
sim_var <- var(searches)

cat("Simulated Mean (Complete Set):", sim_mean, "\n")
```

```
## Simulated Mean (Complete Set): 1.6627
```

```
cat("Simulated Variance (Complete Set):", sim_var, "\n")
```

```
## Simulated Variance (Complete Set): 1.129442
```

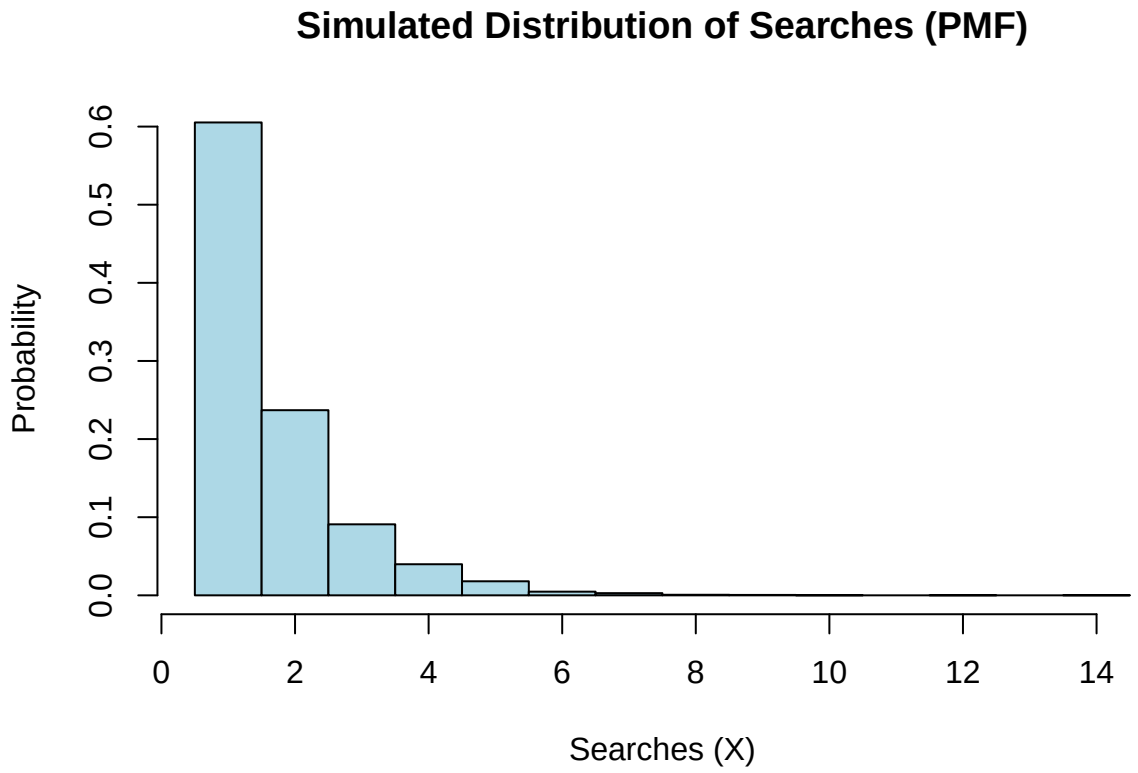
```r
cat("Theoretical Mean (1/p):", 1/p, "\n")
```

```
## Theoretical Mean (1/p): 1.666667
```

```r
cat("Theoretical Variance ((1-p)/p^2):", (1-p)/(p^2), "\n")
```

```
## Theoretical Variance ((1-p)/p^2): 1.111111
```

```r
# Plot Simulated PMF
hist(searches, probability = TRUE, breaks = seq(0.5, max(searches) + 0.5, by = 1),
     col = "lightblue", main = "Simulated Distribution of Searches (PMF)",
     xlab = "Searches (X)", ylab = "Probability")
```




```r
# --- Conditional Distribution Given 3 Initial Failures (X > 3) ---
conditional_searches <- searches[searches > 3] - 3

cond_mean <- mean(conditional_searches)
cond_var <- var(conditional_searches)

cat("\n--- Conditional Distribution (Given X > 3) ---", "\n")
```

```
##
## --- Conditional Distribution (Given X > 3) ---
```

```r
cat("Conditional Mean (Remaining Searches):", cond_mean, "\n")
```

```
## Conditional Mean (Remaining Searches): 1.651198
```

```r
cat("Conditional Variance (Remaining Searches):", cond_var, "\n")
```

```
## Conditional Variance (Remaining Searches): 1.160014
```

```r
# --- Testing the Markov Memoryless Property ---
p_x4_given_x3 <- sum(searches == 4) / sum(searches > 3)
p_x1 <- sum(searches == 1) / n_trials

p_x5_given_x3 <- sum(searches == 5) / sum(searches > 3)
p_x2 <- sum(searches == 2) / n_trials

cat("\n--- Markov Memoryless Property Verification ---", "\n")
```

```
##
## --- Markov Memoryless Property Verification ---
```

```r
cat("(a) P(X = 4 | X > 3):", p_x4_given_x3, "| P(X = 1):", p_x1, "\n")
```

```
## (a) P(X = 4 | X > 3): 0.5958084 | P(X = 1): 0.6053
```

```r
cat("(b) P(X = 5 | X > 3):", p_x5_given_x3, "| P(X = 2):", p_x2, "\n")
```

```
## (b) P(X = 5 | X > 3): 0.2694611 | P(X = 2): 0.237
```

Interpretation Memoryless Property: The simulation verifies the memoryless property of geometric distributions $P(X = k + s \mid X > s) = P(X = k)$. Given that 3 searches have already failed ($X > 3$), the probability that the first success occurs on the 4th total search ($P(X = 4 \mid X > 3) \approx 0.5958$) is equal to the probability of finding it on the very first search ($P(X = 1) \approx 0.6053$).

Conditional Distribution: The conditional mean (r round(cond_mean, 4)) and variance (r round(cond_var, 4)) for remaining searches closely match the complete set simulation mean (r round(sim_mean, 4)) and variance (r round(sim_var, 4)), confirming that past failed attempts do not alter future probabilities.